VSS: An Interactive Search and Sort Algorithm Visualizer

Arya Acharya
Department of Computer Science and Engineering
CHRIST (Deemed to be University)
aryaacharya2500@gmail.com

Prof. Rashmi Jain
Assistant Professor
CHRIST (Deemed to be University)
rashmijain08311@gmail.com

Abstract

This paper presents the design and development of a mobile-based algorithm visualization system implemented for Android devices. The application enables dynamic representation of fundamental sorting and searching techniques using animated graphical structures and synchronized feedback mechanisms.

The framework integrates real-time computational metrics including execution time, memory estimation, and step tracking. An adaptive sound mapping mechanism converts numeric transitions into proportional frequency outputs, reinforcing conceptual understanding through multimodal interaction.

The proposed system establishes a structured learning environment that combines algorithmic analysis with interactive experimentation, offering a portable educational solution suitable for academic contexts.

Index Terms - Algorithm Visualization, Sorting Algorithms, Searching Algorithms, Android Application, Educational Technology, Interactive Learning.

Introduction

Algorithmic reasoning forms the foundation of computer science education. However, conceptual abstraction often limits intuitive understanding, particularly for recursive and iterative processes. Visualization systems provide an effective bridge between theoretical explanation and observable behavior.

The presented work introduces an Android-based visualization platform supporting multiple sorting and searching algorithms. The system demonstrates algorithmic flow, internal state transitions, and performance variation under changing input conditions.

Objectives

The objective of this work is to construct an interactive environment that allows learners to observe and analyze algorithmic behavior in real time. The system integrates execution tracking, graphical animation, and auditory reinforcement within a unified mobile application.

An additional goal is to compare observed performance trends with theoretical complexity expectations while maintaining responsive visual interaction.

Literature Review

Research in algorithm visualization has consistently highlighted its effectiveness in improving conceptual retention. Earlier systems primarily focused on desktop or browser-based implementations, often emphasizing animation without detailed runtime analytics.

Mobile-oriented visualization systems remain relatively limited, particularly those integrating real-time metric computation and auditory mapping. This work adapts established visualization principles to a portable platform while incorporating measurable performance indicators.

Methodology

The system follows a modular design separating algorithm execution, animation handling, metric computation, and sound generation.

Sorting techniques implemented include Radix Sort, Merge Sort, and Tree Sort. Searching techniques include Linear Search, Jump Search, and Tree Search. Each algorithm executes stepwise while dynamically updating graphical bar representations.

Performance metrics are recorded during runtime. Sound frequencies are computed proportionally to numeric magnitude, ensuring synchronized auditory variation during element comparison.

Implementation and Results

Experimental Setup

The application was developed using Unity Engine and C# scripting. Testing was conducted on Android devices with varying input sizes to evaluate performance trends and animation responsiveness.

Performance Metrics

Execution time was measured in milliseconds, step count was recorded for each iteration, and memory usage was approximated using profiling utilities. Observed results aligned with theoretical expectations such as logarithmic growth in divide-and-conquer algorithms and linear growth in sequential search methods.

Observations

Visualization improved traceability of recursive processes. Audio reinforcement enhanced engagement without affecting computational efficiency. Performance trends demonstrated expected behavior as input size increased.

Limitations

Extremely large datasets may reduce animation smoothness. Memory estimation remains approximate. Performance may vary depending on hardware specifications.

Recommendations

Future development may incorporate additional algorithms, shuffle techniques, custom user inputs, and expanded analytical comparison tools.

Code Snippet Integration

```
TreeNode InsertRecursive(TreeNode node, int value, int depth, float x, float spread, TreeNode parent)
{
    if (node == null)
    {
        GameObject obj = Instantiate(nodePrefab, treeContainer);
        RectTransform rt = obj.GetComponent<RectTransform>();
        rt.anchoredPosition = new Vector2(x, -depth * verticalSpacing);
        rt.localScale = Vector3.zero;
        rt.DOScale(Vector3.one, 0.4f).SetEase(Ease.OutBack);
        obj.GetComponentInChildren<TMP_Text>().text = value.ToString();

        TreeNode newNode = new TreeNode(value, rt);
        allNodes.Add(newNode);

        if (parent != null)
            CreateLine(parent, newNode);

        return newNode;
    }

    if (value < node.value)
        node.left = InsertRecursive(node.left, value, depth + 1, x - spread / 2, spread / 2, node);
    else
        node.right = InsertRecursive(node.right, value, depth + 1, x + spread / 2, spread / 2, node);

    return node;
}
```

Figure 1: Sorting Algorithm Implementation

```
1 reference
IEnumerator LinearSearchCoroutine(int target)
{
    float maxVal = 50f;

    AlgorithmMetrics.Instance.StartTracking(data.Length);

    for (int i = 0; i < visualBars.Count; i++)
    {
        AlgorithmMetrics.Instance.AddStep();

        Image barImage = visualBars[i].GetComponent<Image>();

        barImage.DOColor(Color.yellow, 0.3f);
        visualBars[i].transform.DOScale(originalScales[visualBars[i]] * 1.2f, 0.2f);

        int currentVal = int.Parse(visualBars[i].GetComponentInChildren<TMP_Text>().text);

        AlgorithmAudioGenerator.Instance.PlayPing(currentVal, maxVal);

        yield return new WaitForSeconds(0.5f);

        if (currentVal == target)
        {
            barImage.DOColor(Color.green, 0.3f);
            AlgorithmAudioGenerator.Instance.PlaySuccessSound();

            AlgorithmMetrics.Instance.StopTracking();
            yield break;
        }
        else
        {
            barImage.DOColor(Color.red, 0.2f);
            visualBars[i].transform.DOScale(originalScales[visualBars[i]], 0.2f);
            yield return new WaitForSeconds(0.2f);
            barImage.DOColor(Color.white, 0.3f);
        }
    }

    AlgorithmMetrics.Instance.StopTracking();
}
```

Figure 2: Searching Algorithm Implementation

Output Screenshots

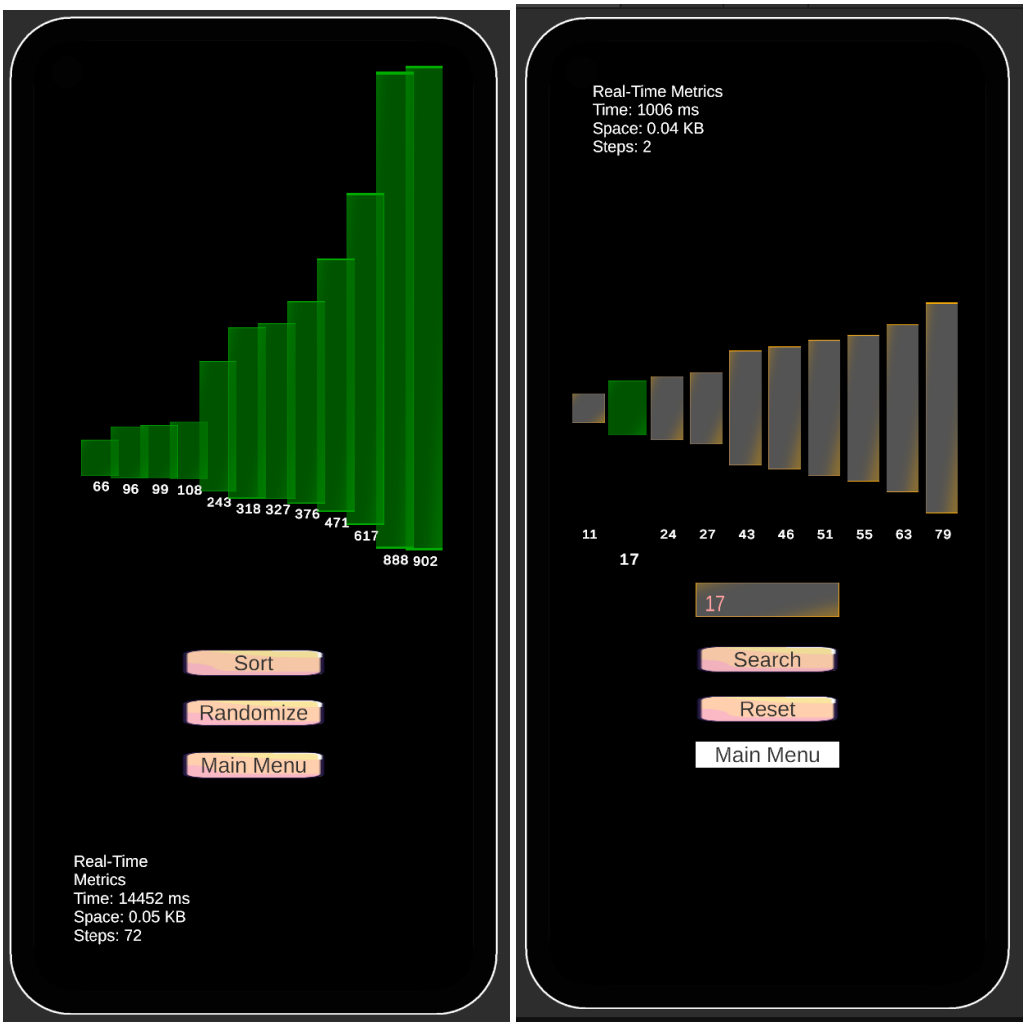

Figure 3: Visualization During Sorting and Searching

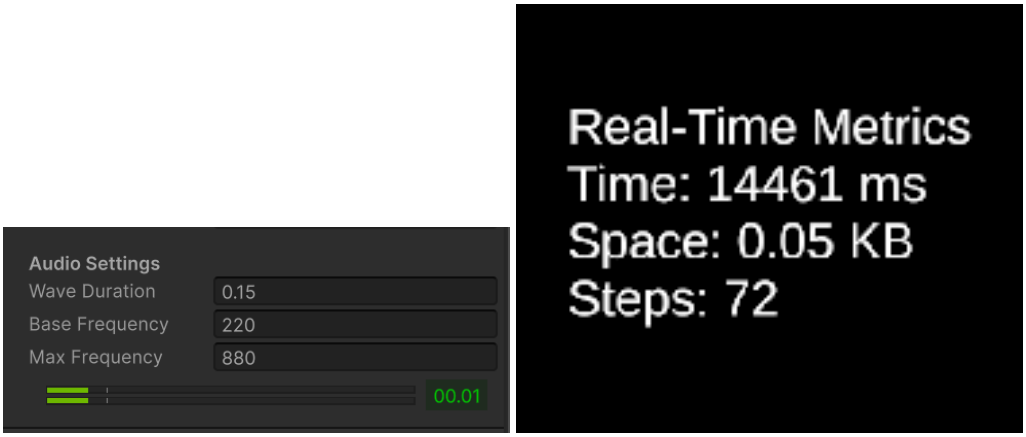

Figure 4: Metrics and Audio Feedback Interface

Conclusion

The developed visualization system demonstrates the feasibility of integrating algorithm animation, metric computation, and auditory feedback within a mobile environment. Observed experimental behavior aligns with theoretical complexity expectations while maintaining interactive clarity. The framework provides a scalable educational platform suitable for further academic enhancement.

References

[1] ACM Digital Library, "Paper." Available: https://dl.acm.org/doi/abs/10.1145/1538234.1538236

[2] ACM Digital Library, "Paper." Available: https://dl.acm.org/doi/abs/10.1145/1821996.1821997

[3] ACM Digital Library, "Paper." Available: https://dl.acm.org/doi/abs/10.1145/1227310.1227366

[4] Google Books, "Visualization of Algorithms." Available: https://books.google.com/books?hl=en&lr=&id=Q-cxEQAAQBAJ&oi=fnd&pg=PA199&dq=visualization+of+algorithms

[5] IEEE Xplore, "Paper." Available: https://ieeexplore.ieee.org/abstract/document/9552925/

[6] C. Hundhausen, S. Douglas, and J. Stasko, "A Meta-Study of Algorithm Visualization Effectiveness," Journal of Visual Languages and Computing, 2002. Available: https://doi.org/10.1006/jvlc.2002.0232

[7] ACM Digital Library, "Algorithm Visualization: A Report on the State of the Field." Available: https://doi.org/10.1145/1227504.1227366

[8] ACM Transactions on Computing Education, "Article." Available: https://www.tandfonline.com/doi/abs/10.1080/08993408.2016.1160664

[9] International Journal of Modern Education and Computer Science, "Paper." Available: https://www.academia.edu/download/38398785/3111ijma03.pdf

[10] ICTERI 2019 Proceedings, "Conference Paper." Available: https://www.researchgate.net/profile/Vadim-Ermolayev/publication/334121888_Proc_15th_Int_Conf_on_ICT_in_Education_Research_and_Industrial_Applications_ICTERI_2019_Volume_I_Main_Conference_Kherson_Ukraine_June_12-15_2019/links/5d1dc5f292851cf440633110/Proc-15th-Int-Conf-on-ICT-in-Education-Research-and-Ind

ustrial-Applications-ICTERI-2019-Volume-I-Main-Conferenc
e-Kherson-Ukraine-June-12-15-2019.pdf